Chap.2 Assemblers

수업 목표

- Assembler의 알고리즘과 자료구조의 기본 개념 및 기능에 대해 이해한다.
- 예제를 통해 기본 개념과 기능 학습을 확인한다.

Assembler Tables and Logic (I)

- Our simple assembler uses two internal tables:
  - The OPTAB and SYMTAB.
  - OPTAB is used to look up mnemonic operation codes and translate them to their machine language equivalents.
    - LDA→00, STL→14, …
  - SYMTAB is used to store values (addresses) assigned to labels.
    - FIRST→1000, COPY→1000, …

Assembler Tables and Logic (II)

- Location Counter (LOCCTR)
  - A variable used to help in the assignment of addresses
  - Initialized to the beginning address specified in the START statement
  - After each source statement is processed, the length of assembled instruction or data area to be generated is added to LOCCTR
  - Whenever we reach a label in the source program, the current value of LOCCTR gives the address to be associated with that label

Assembler Tables and Logic (III)

- Operation Code Table (OPTAB)
  - Contain the mnemonic operation code & its machine language equivalent.
    - May also contain information about instruction format and length
  - Used to look up & validate mnemonic operation codes (Pass 1) and translate them to machine language (Pass 2)
    - In SIC, both processes could be done together
    - In SIC/XE, search OPTAB to find the instruction length (Pass 1), determine instruction format to use in assembling the instruction (Pass 2)
  - Usually organized as a hash table & a static table
    - Mnemonic operation code as a key
    - Provide fast retrieval with a minimum searching
    - A static table in most cases
      - Entries are not normally added to or deleted from it

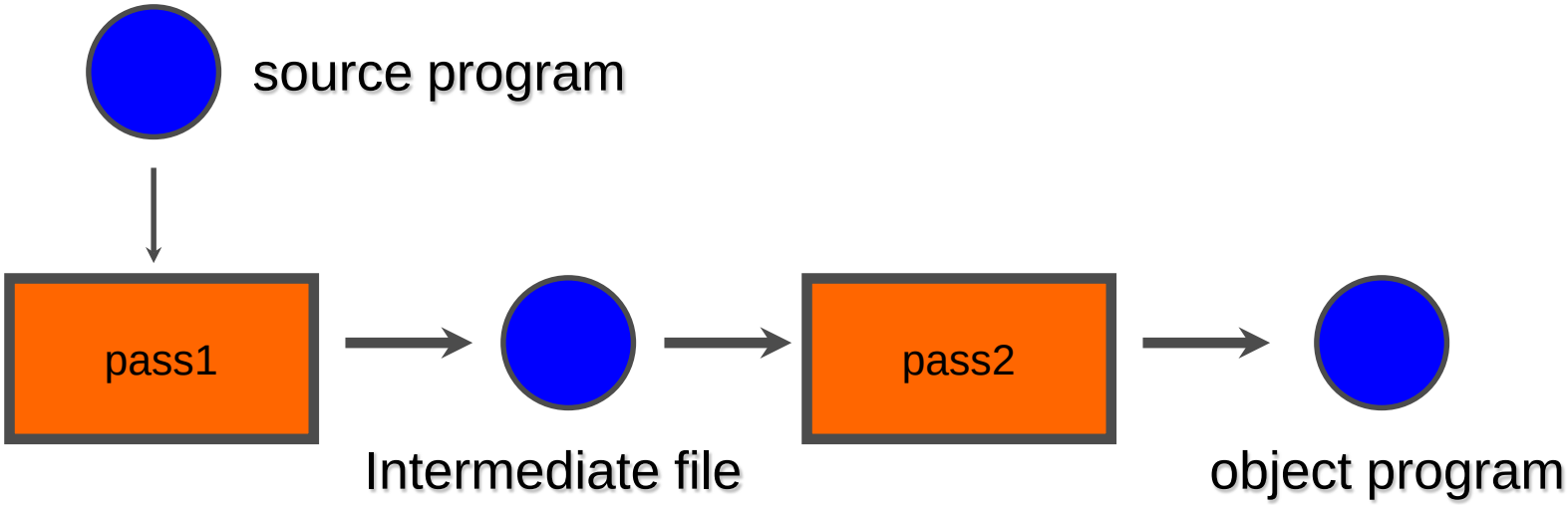
Assembler Tables and Logic (IV)

- Symbol Table (SYMTAB)
  - Used to store values (addresses) assigned to labels
  - Contain the name and value (addresses) for each label, together with flags for error conditions, also information about the data area or instruction labeled
    - for example, its the type or length
  - Pass 1 : labels are entered with their assigned (from LOCCTR)
  - Pass 2 : symbols are used to look up in SYMTAB to obtain the addresses to be inserted
  - Usually organized as a hash table
    - For efficiency of insertion & retrieval
    - Heavily used throughout the assembly

Assembler Tables and Logic (v)

- Intermediate file
  - to communicate between the two passes
  - written by Pass 1, and used as the input to Pass 2
  - containing each source statement
    together with its assigned address, error indicators, etc

Logic Diagram



(containing each source statement
together with its assigned address,
error indicators, etc.)

Algorithm for Pass 1 (Fig 2.4(a)-I)

```
Pass 1:

begin
   read first input line
   if OPCODE = 'START' then
      begin
        save #[OPERAND] as starting address
        initialize LOCCTR to starting address
        write line to intermediate file
        read next input line
      end {if START}
   else
      initialize LOCCTR to 0
   while OPCODE ≠'END' do
      begin
        if this is not a comment line then
           begin
              if there is a symbol in the LABEL field then
                 begin
                    search SYMTAB for LABEL
                    if found then
                        set error flag(duplicate symbol)
                    else
                        insert (LABEL, LOCCTR) into SYMTAB
                 end {if symbol}
```

Algorithm for Pass 1 (Fig 2.4(a)-II)

```
                search OPTAB for OPCODE
                if found then
                   add 3 {instruction length} to LOCCTR
                else if OPCODE = 'WORD' then
                   add 3 to LOCCTR
                else if OPCODE = 'RESW' then
                   add 3 * #[OPERAND] to LOCCTR
                else if OPCODE = 'RESB' then
                   add #[OPERAND] to LOCCTR
                else if OPCODE = 'BYTE' then
                   begin
                      find length of constant in bytes
                      add length to LOCCTR
                   end {if BYTE}
                else
                   set error flag (invalid operation code)
             end {if not a comment)
          write line to intermediate file
          read next input line
       end {while not END}
    write last line to intermediate file
    save (LOCCTR - starting address) as program length
end {Pass 1}
```

Output of Pass 1

- SYMTAB

FIRST	1000
CLOOP	1003
ENDFIL	1015
EOF	102A
THREE	102D
ZERO	1030
RETADR	1033
LENGTH	1036
BUFFER	1039

RDREC	2039
RLOOP	203F
EXIT	2057
INPUT	205D
MAXLEN	205E
WRREC	2061
WLOOP	2064
OUTPUT	2079

Program Length = 207A - 1000 = 107A

Algorithm for Pass 2 (Fig 2.4(b)-I)

```
Pass 2:

begin
  read first input line {from intermediate file}
  if OPCODE = 'START' then
     begin
        write listing line
        read next input line
     end {if START}
  write Header record to object program
  initialize first Text record
  while OPCODE ≠'END' do
     begin
        if this is not a comment line then
           begin
              search OPTAB for OPCODE
              if found then
                 begin
                    if there is a symbol in OPERAND field then
                       begin
                          search SYMTAB for OPERAND
                          if found then
                             store symbol value as operand address
                          else
```

Algorithm for Pass 2 (Fig 2.4(b)-II)

```
                                begin
                                   store 0 as operand address
                                   set error flag (undefined symbol)
                                end
                             end {if symbol}
                          else
                             store 0 as operand address
                          assemble the object code instruction
                       end {if opcode found}
                    else if OPCODE = 'BYTE' or 'WORD' then
                       convert constant to object code
                    if object code will not fit into the current Text record then
                       begin
                          write Text record to object program
                          initialize new Text record
                       end
                    add object code to Text record
                 end {if not comment}
              write listing line
              read next input line
           end {while not END}
       write last Text record to object program
       write End record to object program
       write last listing line
end {Pass 2}
```

Output of Pass 2

```
HCOPY  00100000107A
 ^     ^     ^

T0010001E141033482039001036281030301015482061 3C100300102A0C103900102D
 ^     ^ ^     ^     ^     ^     ^     ^     ^     ^     ^     ^

T00101E150C10364820610810334C0000454F46000003000000
 ^     ^ ^     ^     ^     ^     ^     ^     ^

T0020391E041030001030E0205D30203FD8205D2810303020575490392C205E38203F
 ^     ^ ^     ^     ^     ^     ^     ^     ^     ^     ^     ^

T0020571C1010364C0000F1001000041030E02079302064509039DC20792C1036
 ^     ^ ^     ^     ^ ^     ^     ^     ^     ^     ^     ^

T002073073820644C000005
 ^     ^ ^     ^     ^

E001000
 ^
```

이 동영상 교과목 자료는
GJ-RIP 사업의
지자체-대학 협력기반 지역혁신 사업비로 개발되었음.